

Curso: Ciência da Computação

Aplicativos Móveis

Aula 05

ImageView e acesso a câmera

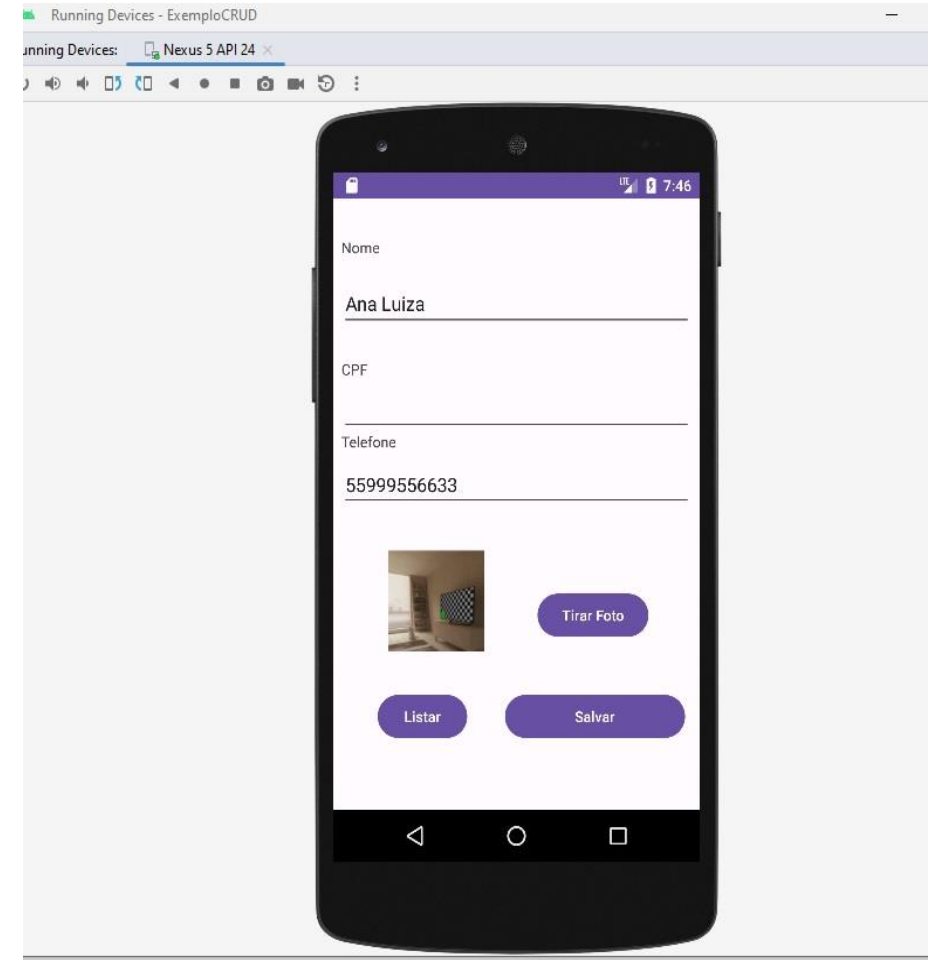
Professor: André Flores dos Santos



ImageView-Botão tirarFoto

Após terminarmos as operações CRUD do nosso projeto com validações de dados, vamos inserir um 'ImageView' e um botão 'tirarFoto' para tirar uma foto na tela de cadastrar o aluno (MainActivity.java)

Inicialmente carregue um objeto 'imageView' ancore ele na tela e escolha o 'id' para ele.

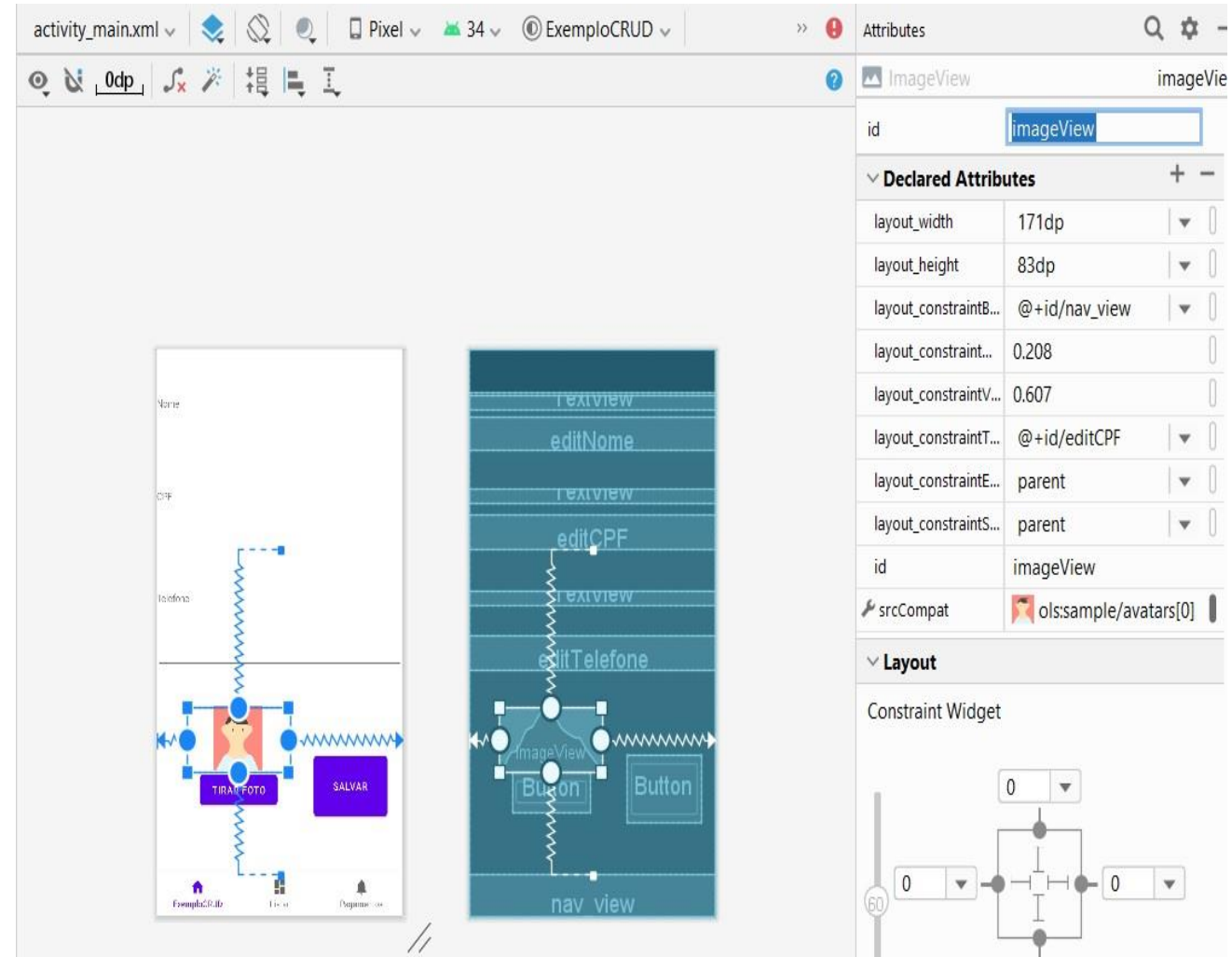


ImageView-Botão tirarFoto

Vamos inserir um ‘ImageView’ e um botão ‘tirarFoto’ para tirar uma foto na tela de ‘cadastrarAluno’.

Inicialmente carregue um objeto ‘imageView’ ancore ele na tela e escolha o **‘id=imageView’** para ele.

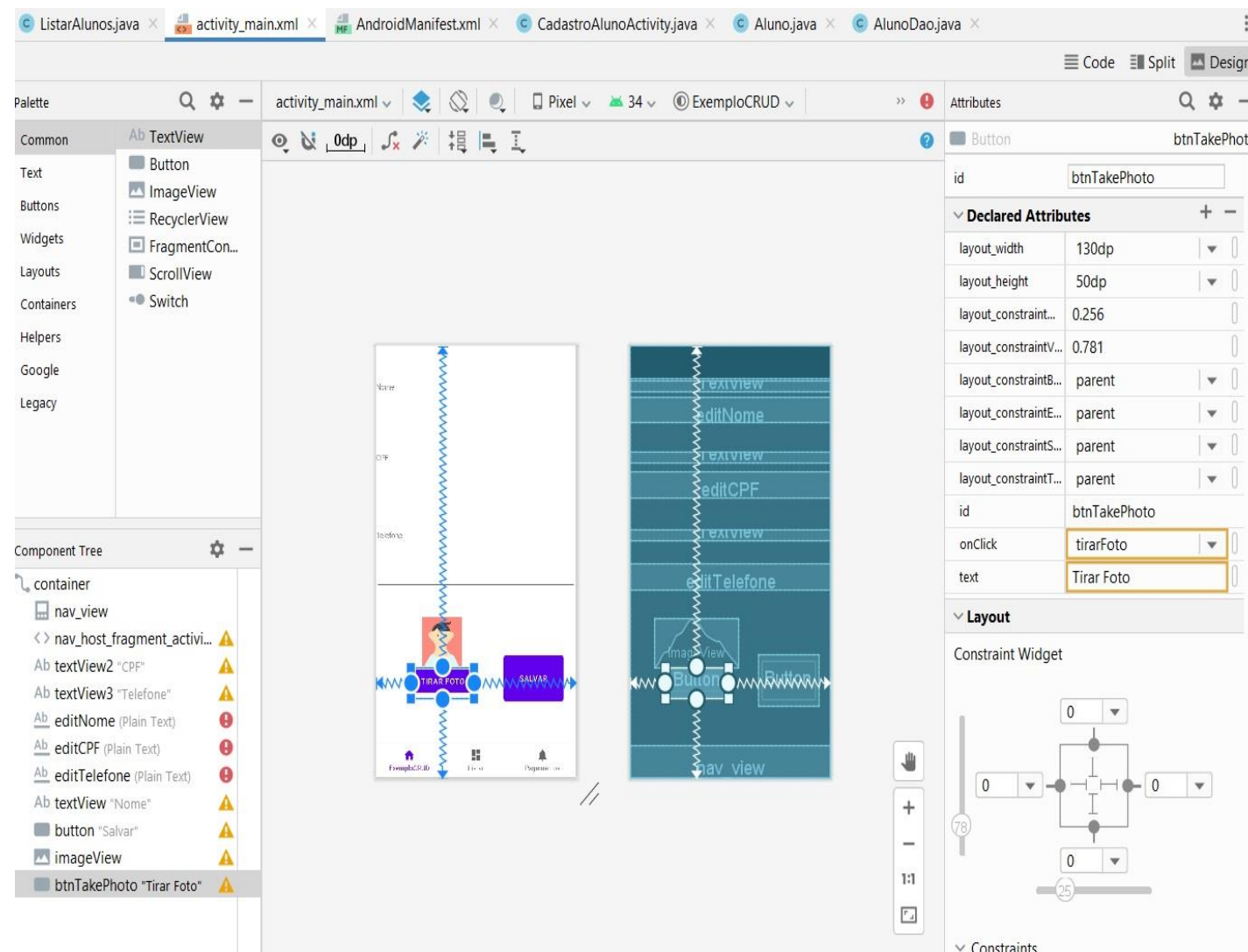
Na opção ‘Pick a Resource’ escolha ‘avatars’ para ficar uma imagem de fundo e de ‘ok’.



ImageView-Botão tirarFoto

Em seguida inserir um botão com
‘id=btnTakePhoto’, **‘text=Tirar Foto’**
e já adicionamos no método
‘onClick=tirarFoto’ que iremos
implementar na sequência.

Ancorar o botão para ficar tudo certo
nesta parte do layout. Verificar no
código xml se o método onClick foi
realmente inserido.



ImageView-Botão tirarFoto

Inicialmente vamos
completar os dados da foto
na classe **Aluno.java**

```
17 usages
public class Aluno implements Serializable {
    2 usages
    private Integer id;
    3 usages
    private String nome;
    2 usages
    private String cpf;
    2 usages
    private String telefone;

    //CAMERA
    2 usages
    private byte[] fotoBytes;

    3 usages
    public byte[] getFotoBytes() { return fotoBytes; }

    2 usages
    public void setFotoBytes(byte[] fotoBytes) { this.fotoBytes = fotoBytes; }

    //CAMERA

    public Integer getId() { return id; }

    public void setId(Integer id) { this.id = id; }
```

ImageView-Botão tirarFoto

Devemos completar
o método **inserir()**
com os dados da foto
na classe
AlunoDao.java

```
//-----método para inserir-----//  
1 usage  
public long inserir(Aluno aluno){ // long porque retorna o id do aluno  
    if (!cpfExistente(aluno.getCpf()) ) {  
        ContentValues values = new ContentValues(); //valores que irei inserir  
        values.put("nome", aluno.getNome());  
        values.put("cpf", aluno.getCpf());  
        values.put("telefone", aluno.getTelefone());  
        values.put("fotoBytes", aluno.getFotoBytes()); // FOTO  
        return banco.insert( table: "aluno", nullColumnHack: null, values); //tabela aluno, não tera colunas vazias, valores values  
    }  
    else{  
        // CPF já existe, você pode lidar com isso de acordo com sua lógica  
        return -1; // Retorna -1 para chamada do método inserir() no CadastroAlunoActivity  
    }  
}
```


ImageView-Botão tirarFoto

Devemos completar o método `atualizar()` com os dados da foto na classe `AlunoDao.java`

```
//----- ATUALIZAR -----//  
1 usage  
public void atualizar(Aluno aluno){  
    ContentValues values = new ContentValues(); //valores que irei inserir  
    values.put("nome", aluno.getNome());  
    values.put("cpf", aluno.getCpf());  
    values.put("telefone", aluno.getTelefone());  
    values.put("fotoBytes", aluno.getFotoBytes()); // FOTO  
    banco.update( table: "aluno", values, whereClause: "id = ?", new String[]{aluno.getId().toString()});  
}
```

ImageView-Botão tirarFoto

Devemos
completar o
método
obterTodos() com
os dados da foto
na classe
AlunoDao.java

```
//-----CONSULTAR ALUNOS-----//  
  
2 usages  
  
public List<Aluno> obterTodos(){  
    List<Aluno> alunos = new ArrayList<>();  
    Log.d( tag: "AlunoDao", msg: "obterTodos" + alunos.size() + 1 + "Alunos Encontrados: "); //Log no terminal  
  
    //cursor aponta para as linhas retornadas  
    Cursor cursor = banco.query( table: "aluno", new String[]{"id", "nome", "cpf", "telefone", "fotoBytes"},  
                                selection: null, selectionArgs: null, groupBy: null, having: null, orderBy: null); //nome da tabela, nome das colunas, completa com null o método  
    //que por padrão pede esse número de colunas obrigatórias  
    while(cursor.moveToNext()){ //verifica se consegue mover para o próximo ponteiro ou linha  
        Aluno a = new Aluno();  
        a.setId(cursor.getInt( columnIndex: 0)); // new String[]{"id", "nome", "cpf", "telefone"}, id é coluna '0'  
        a.setNome(cursor.getString( columnIndex: 1)); // new String[]{"id", "nome", "cpf", "telefone"}, nome é coluna '1'  
        a.setCpf(cursor.getString( columnIndex: 2)); // new String[]{"id", "nome", "cpf", "telefone"}, cpf é coluna '2'  
        a.setTelefone(cursor.getString( columnIndex: 3)); // new String[]{"id", "nome", "cpf", "telefone"}, telefone é coluna '3'  
        //FOTO  
        a.setFotoBytes(cursor.getBlob( columnIndex: 4)); // new String[]{"id", "nome", "cpf", "telefone", "fotoBytes"}, fotoBytes é coluna '4'  
        alunos.add(a);  
    }  
    return alunos;  
}
```


ImageView-Botão tirarFoto

No arquivo

AndroidManifest.xml

, devemos configurar algumas linhas de permissões para usar a câmera

```

1  <?xml version="1.0" encoding="utf-8"?>
2  <manifest xmlns:android="http://schemas.android.com/apk/res/android"
3          xmlns:tools="http://schemas.android.com/tools">
4
5      <!-- premissões para acessar a camera -->
6      <uses-feature android:name="android.hardware.camera" />
7      <uses-permission android:name="android.permission.CAMERA" />
8      <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE" />
9      <!-- premissões para acessar a camera -->
10
11     <application
12         android:allowBackup="true"
13         android:dataExtractionRules="@xml/data_extraction_rules"
14         android:fullBackupContent="@xml/backup_rules"
15         android:icon="@mipmap/ic_launcher"
16         android:label="ExemploCRUD"
17         android:roundIcon="@mipmap/ic_launcher_round"
18         android:supportsRtl="true"
19         android:theme="@style/Theme.ExemploCRUD"
20         tools:targetApi="31">
21         <activity
22             android:name=".ListarAlunos"

```

ImageView-Botão tirarFoto

Agora vamos fazer as configurações na Classe de **'MainActivity.java'** ou o nome que vocês utilizaram para a classe cadastrar aluno.

Declarações antes do método **onCreate()**

```
/*-----CONFIGURACOES PARA CAMERA-----*/  
private ImageView imageView; // Adicione a ImageView para exibir a foto tirada  
  
// Código de identificação para a solicitação de PERMISSÃO da câmera.  
// É usado no método 'tirarFoto' para pedir autorização e  
// verificado no 'onRequestPermissionsResult' para saber se o usuário aceitou.  
private static final int CAMERA_PERMISSION_CODE = 100;  
  
// Código de identificação para a CAPTURA DA IMAGEM (a foto em si).  
// É usado no método 'startCamera' ao iniciar a intenção da câmera e  
// verificado no 'onActivityResult' para garantir que os dados recebidos são da foto.  
private static final int REQUEST_IMAGE_CAPTURE = 200;
```

- **Dicas de tipos de variáveis:**

- **private** → Protege o acesso externo.

- **static** → O valor pertence à classe, não a objetos individuais.

- **final** → O valor não pode ser modificado depois de definido.

ImageView-Botão tirarFoto

Agora dentro do método `onCreate` da classe `MainActivity.java`, vamos vincular os campos dos objetos.

```
// CAMERA - Vincula a ImageView no layout para mostrar a foto  
imageView = findViewById(R.id.imageView);  
  
// Vincula o botão do layout com a variável 'btnTakePhoto'.  
// Este botão será utilizado para disparar o evento de captura de foto quando clicado.  
Button btnTakePhoto = findViewById(R.id.btnTakePhoto);
```

ImageView-Botão tirarFoto

Deixar organizado o código com a intenção 'extra' do método de atualizar os dados, dentro do `onCreate()` do `MainActivity.java`.

Colocamos para carregar a foto no momento que chamar o atualizar.

O método `getFotosBytes()` os dados da foto devem ser atualizados

```
//-- Código que recebe os dados do método Atualizar() se ele estiver enviando algo  
//(ListarAlunosActivity)-----  
Intent it = getIntent(); //pega intenção  
if(it.hasExtra("aluno")){  
    aluno = (Aluno) it.getSerializableExtra("aluno");  
    nome.setText(aluno.getNome().toString());  
    cpf.setText(aluno.getCpf());  
    telefone.setText(aluno.getTelefone());  
  
    // Carregar a foto no ImageView no momento que carregar os dados para atualizar  
    byte[] fotoBytes = aluno.getFotosBytes();  
    if (fotoBytes != null && fotoBytes.length > 0) {  
        Bitmap bitmap = BitmapFactory.decodeByteArray(fotoBytes, 0, fotoBytes.length);  
        imageView.setImageBitmap(bitmap);  
    }  
}
```

ImageView-Botão tirarFoto

Atualizar a classe

Conexao.java, com os dados da foto para serem salvas no banco de dados.

OBS: Desinstalar o app para o bd antigo ser retirado, **ou** atualizar o método **onUpgrade** se quiser manter os dados que já tinham (próximo slide)

```
public class Conexao extends SQLiteOpenHelper {

    private static final String name = "banco.db";
    private static final int version = 1;

    public Conexao(Context context){
        super(context, name, null, version);
    }

    @Override
    public void onCreate(SQLiteDatabase db) {
        /*db.execSQL("create table aluno(id integer primary key autoincrement, " +
            "nome varchar(50), cpf varchar(50), telefone varchar(50));*/
        db.execSQL("create table aluno(id integer primary key autoincrement, " + "nome varchar(50), " +
            "cpf varchar(50), telefone varchar(50), foto_bytes BLOB)");
    }

    @Override
    public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {

    }
}
```

ImageView-Botão tirarFoto

(opcional) Atualizar o método onUpgrade para não perder os dados que já estavam no banco de dados na classe

Conexao.java.

```
public class Conexao extends SQLiteOpenHelper {

    private static final String name = "banco.db";
    private static final int version = 2;

    public Conexao(Context context){
        super(context, name, null, version);
    }

    @Override
    public void onCreate(SQLiteDatabase db) {
        db.execSQL("create table aluno(id integer primary key autoincrement, " + "nome varchar(50), " +
            "cpf varchar(50), telefone varchar(50))");
    }

    @Override
    public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
        if (oldVersion < 2) {
            // Adiciona a coluna fotoBytes se estiver atualizando da versão 1 para 2
            db.execSQL("ALTER TABLE aluno ADD COLUMN fotoBytes BLOB");
        }
    }
}
```


ImageView-Botão tirarFoto

Inserir o método do botão
'tirarFoto', dentro da
MainActivity.java.

```
//-----MÉTODOS DA CAMERA -----  
/**  
 * Método chamado pelo clique do botão "Tirar Foto" (android:onClick no XML).  
 * Sua principal função é verificar se o usuário já permitiu o uso da câmera.  
 */  
public void tirarFoto(View view) {  
    // Verifica se a permissão de CAMERA já foi concedida anteriormente  
    if (ContextCompat.checkSelfPermission(this, Manifest.permission.CAMERA) != PackageManager.PERMISSION_GRANTED) {  
  
        // Se NÃO tem permissão, abre a janelinha do sistema pedindo a autorização.  
        // O código CAMERA_PERMISSION_CODE (100) serve para identificarmos esta resposta depois.  
        ActivityCompat.requestPermissions(this, new String[]{Manifest.permission.CAMERA}, CAMERA_PERMISSION_CODE);  
    } else {  
        // Se JÁ tem permissão, chama o método para abrir a câmera de fato.  
        startCamera();  
    }  
}
```

ImageView-Botão tirarFoto

Inserir o método `onRequestPermissionsResult()` dentro da `MainActivity.java` para verificar as se permissões da câmera foram aceitas pelo usuário.

```
/** -----  
 * Método chamado automaticamente após o usuário responder à solicitação de permissão.  
 * @param requestCode O código (100) que enviamos lá no tirarFoto.  
 * @param permissions O array de permissões solicitadas.  
 * @param grantResults O resultado (concedido ou negado) para cada permissão.  
 */  
@Override  
public void onRequestPermissionsResult(int requestCode, @NonNull String[] permissions, @NonNull int[] grantResults) {  
    super.onRequestPermissionsResult(requestCode, permissions, grantResults);  
    // Verifica se a resposta que chegou é referente ao nosso pedido de câmera (código 100)  
    if (requestCode == CAMERA_PERMISSION_CODE) {  
        // Verifica se o array de resultados não está vazio e se o usuário clicou em "Permitir"  
        if (grantResults.length > 0 && grantResults[0] == PackageManager.PERMISSION_GRANTED) {  
            Log.d("CAMERA_DEBUG", "Usuário permitiu, abrindo câmera...");  
            startCamera();  
        } else {  
            // Se o usuário negou, avisamos que ele não conseguirá tirar fotos.  
            Toast.makeText(this, "A permissão é necessária para usar a câmera.", Toast.LENGTH_SHORT).show();  
        }  
    }  
}
```

ImageView-Botão tirarFoto

Inserir o método 'startCamera()', dentro da 'MainActivity.java'.

```
/** -----  
 * Método auxiliar que cria a "Intenção" de abrir o aplicativo de câmera do dispositivo.  
 */  
private void startCamera() {  
    try {  
        // Cria uma Intent (intenção) para capturar uma imagem.  
        Intent takePictureIntent = new Intent(MediaStore.ACTION_IMAGE_CAPTURE);  
  
        // Inicia a atividade da câmera esperando um resultado (a foto).  
        // O código REQUEST_IMAGE_CAPTURE (200) serve para identificarmos esta foto quando ela voltar.  
        startActivityForResult(takePictureIntent, REQUEST_IMAGE_CAPTURE);  
    } catch (Exception e) {  
        Log.e("CAMERA_DEBUG", "Erro ao abrir a câmera: " + e.getMessage());  
        Toast.makeText(this, "Erro ao abrir a câmera no seu dispositivo.", Toast.LENGTH_SHORT).show();  
    }  
}
```

ImageView-

Botão tirarFoto

Método

‘onActivityResult’ da
‘MainActivity.java’.

Verifica a atividade
de retorno da câmera.

** Método chamado quando uma atividade que iniciamos (como a câmera) termina e nos devolve dados.*

@Override

```
protected void onActivityResult(int requestCode, int resultCode, Intent data) {
    super.onActivityResult(requestCode, resultCode, data);
    // 1. Verifica se o resultado que está voltando é o da nossa Câmera (código 200)
    // 2. Verifica se a foto foi tirada com sucesso (RESULT_OK)
    if (requestCode == REQUEST_IMAGE_CAPTURE && resultCode == RESULT_OK) {
        // Extrai a imagem compactada (thumbnail) que vem dentro da Intent 'data'
        Bundle extras = data.getExtras();
        Bitmap imageBitmap = (Bitmap) extras.get("data");
        // Exibe a foto na tela para o usuário ver
        imageView.setImageBitmap(imageBitmap);

        // --- PREPARAÇÃO PARA O BANCO DE DADOS ---
        // Transforma o Bitmap em um Array de Bytes (byte[]), que é como o SQLite guarda imagens (BLOB)
        ByteArrayOutputStream stream = new ByteArrayOutputStream();
        imageBitmap.compress(Bitmap.CompressFormat.PNG, 100, stream);
        byte[] byteArray = stream.toByteArray();

        // Se o objeto 'aluno' for nulo (novo cadastro), instanciamos ele aqui para guardar a foto
        if (aluno == null){
            aluno = new Aluno();
        }
        // Guarda os bytes da foto no objeto aluno (será usado no método salvar())
        aluno.setFotoBytes(byteArray);
    }
}
```

ImageView-Botão tirarFoto

Configuração que estamos usando no MainActivity.java (cadastrar aluno) para verificar se é um cadastro novo ou é 'atualizar' aluno.

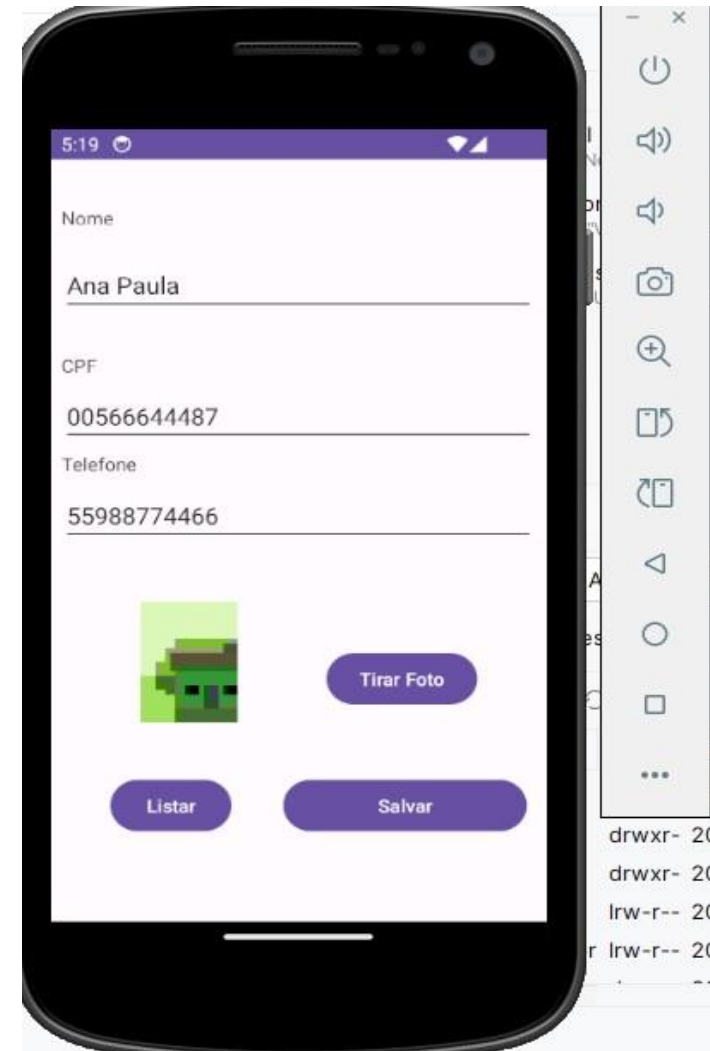
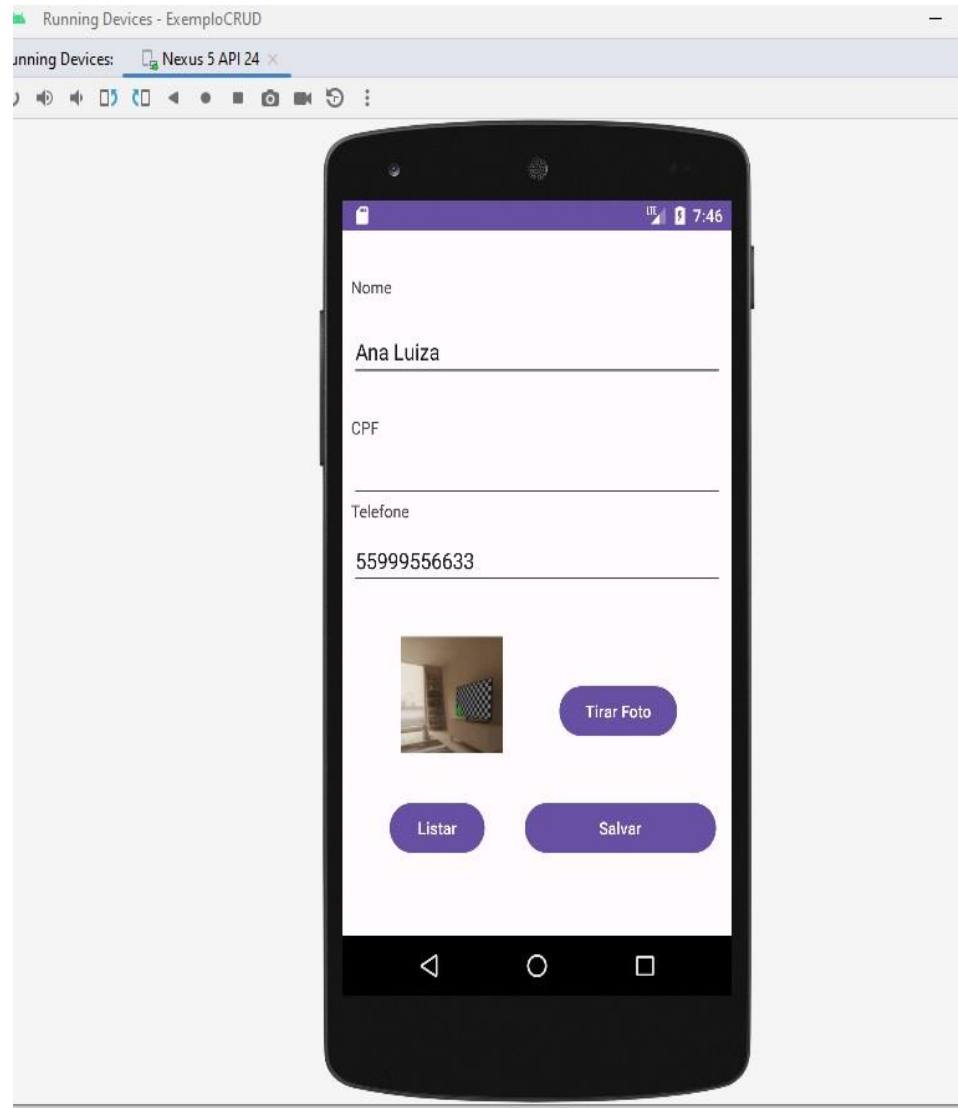
```
// aluno ==null cadastrar, aluno!=null esta recebendo do ListarAlunos
if (aluno==null || aluno.getId() == null) {
    /// CADASTRO DE NOVO ALUNO

    //Se o aluno não tirou foto, o objeto 'aluno' ainda é nulo.
    if (aluno==null){
        Aluno aluno = new Aluno();
    }

    aluno.setNome(nomeDigitado);
    aluno.setCpf(cpfDigitado);
    aluno.setTelefone(telefoneDigitado);
    //Foto ja foi preparada no onActivityResult se aluno tirou a foto
```

```
else{
    // Atualização de um aluno existente se (aluno!=null), significa que esta recebendo
    dados pelo serial do ListarAlunos
    aluno.setNome(nomeDigitado);
    aluno.setCpf(cpfDigitado);
    aluno.setTelefone(telefoneDigitado);
    //Foto ja foi preparada no onActivityResult
```

Resultado



Exercícios:

Entregar até o final da aula, via link do Github o Projeto.

Referências:

- LECHETA, Ricardo R. Google Android: aprenda a criar aplicações para dispositivos móveis com o android SDK. São Paulo: Novatec, 2009.
- TALUKDER, Asoke; YAVAGAL, Roopa. Mobile computing. New Delhi - India: McGraw-Hill, 2007.
- SILVA, D. Desenvolvimento para dispositivos móveis. Pearson, 2017.
- ANDROID. Android Developers. Disponível em <https://developer.android.com>. Acesso em agosto de 20018. 2018.
- MIKKONEN, T. Programming mobile devices: an introduction for practitioners. Chichester, England: Wiley, 2007.
- ROGERS, Rick et al. Desenvolvimento de aplicações Android. O'Reilly: Novatec, 2009.
- SILVA, D. Arquitetura para computação móvel. Pearson, 2018.
- YAVAGAL, Roopa R.; TALUKDER, Asoke K. Mobile computing: technology, applications, and service creation. New York, NY: McGraw-Hill, 2007.

Obrigado pela atenção!!



Contato:
Email: andre.flores@ufn.edu.br

